

Technology Use Statement

1. Which AI-assisted tools did you use? & 2. What did you use each tool for?

- **Claude (agentic mode)** — used throughout the implementation and debugging process.
- **Claude (chat)** — used mainly as a reasoning and writing assistant, to discuss the agent architecture, design testing scenarios, and organize my notes into clearer documentation.

3. How did the tools influence your approach, code, testing, or documentation?

The development process was iterative. I first built a simple loop: probe a few points, send them to the model, check the answer, repeat. As I wanted the agent to get more out of each call, I added more decision paths; a router that analyses the black box locally before any call, free filtering of candidates, and a scheduler that sets aside a function that has stopped producing new information so I restructured the loop into separate modules with an explicit state object per function.

Before using AI tools, I usually debugged with print statements, different inputs, and web searches, which often took several rounds of trial and error. With Claude I could directly provide an error and the relevant code and ask it to investigate or suggest a focused test. Reaching a running prototype quickly mattered more than getting it right the first time, because most of the real problems only became visible once something was actually running.

Testing was influenced the most. The assignment has a €5 cap, so I made the LLM backend swappable and wrote a mock one: `--mock` runs the whole loop offline, and the probing, filtering and error handling were finished before I made a paid call. My total spend was about \$0.03.

For documentation, I organized the key points myself and used AI to turn them into more natural text.

4. How did you review, test, or modify AI-generated suggestions?

For larger changes, I first gave Claude my requirements and step-by-step instructions, then asked it to summarize its understanding and propose a concrete modification plan. I reviewed the plan before asking it to implement the changes, and then reviewed and tested the resulting code myself, so I can retain ownership of the design and implementation decisions.

I tested both normal execution and failure cases. Examples included:

- Running the agent against seventeen functions instead of only the six targets, including several I added specifically to defeat one of my own detectors.
- Printing the state of each function through the observations, the refuted candidates, the evidence sent to check that new information is appended correctly and that nothing the model should not see leaks into the prompt.

5. Examples of incorrect, incomplete, or unsuitable AI-generated suggestions

Verification that proved nothing

The first version checked each candidate on the same points I had already sent to the LLM. I was suspicious of how easily candidates passed, so I asked whether the logic was circular. It was — the model has already seen those values, so of course its expression reproduces them. I changed it to keep points that never enter a prompt.

That was still not enough. The held-out set was a fixed list, and the counterexamples I fed back to the model came from that same list, so each round quietly used up part of the set while the code kept treating

it as clean. The verification points are now generated fresh on every check.

A detector that was wrong twice, in opposite directions

The periodicity test first reported that everything was periodic: after removing a linear trend, a quadratic or a logarithm still leaves a smooth residual whose FFT peak lands in the lowest bin, which reads as a period equal to the whole window. The obvious fix reject long periods, then made it report that nothing was periodic, because the FFT resolution is too coarse to land on 2π exactly. The working version needed three parts together: reject periods longer than a quarter of the window, refine T by local search around the FFT proposal, and confirm that $f(x+T) - f(x)$ is constant rather than zero, so a linear trend is tolerated.

6. Representative prompts or instructions

Examples of instructions I gave to Claude include:

"Is there a circularity problem in this verification logic? If the points used for verification are the same ones fed to the LLM, what does "verification passed" actually tell us?"

"The finite-difference test cannot be trusted on its own, because every smooth function is locally polynomial. Change it step by step:

1. Keep the difference table as a proposal, not a conclusion.
2. Interpolate a polynomial of that degree through $\text{deg}+1$ points.
3. Predict a far point that took no part in the fit, and reject the degree if it misses.

Show me the plan before changing the code."